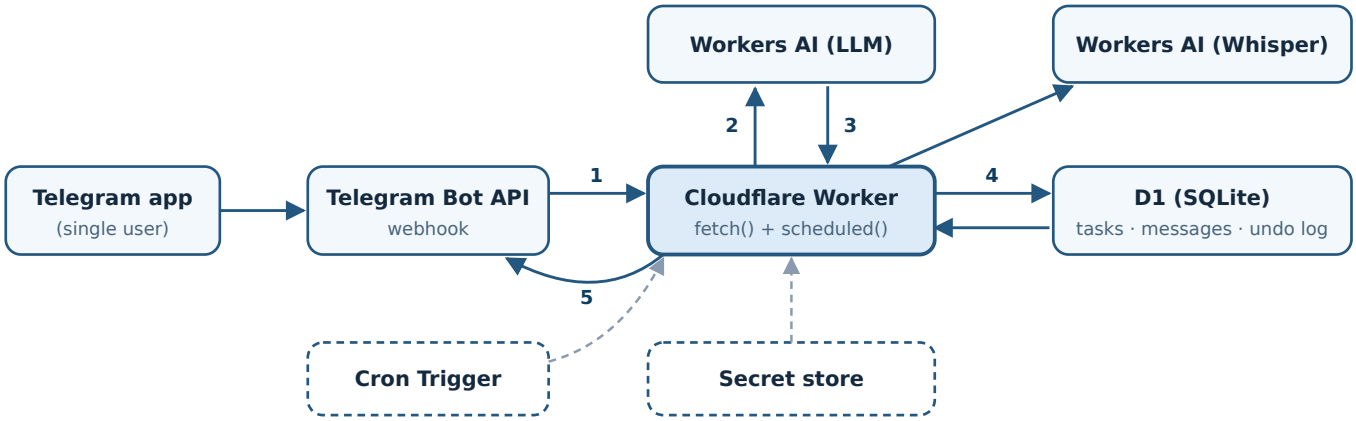
گزارش تحلیلی پروژهٔ رایانش ابری

hermes-mini — دستیار تلگرامیِ بدون سرور روی Cloudflare Workers و D1

نام و نام خانوادگی: شادی مرادی	**درس:** رایانش ابری
آدرس زنده: `https://hermes-mini.shadi-hermes.workers.dev`	**تاریخ:** 11 آبان 1405
	مخزن: `github.com/Shadim09/hermes-mini`

۱. معماری نهایی

ربات پیام فارسیِ آزاد را از تلگرام می‌گیرد و به یک مدل زبانی می‌دهد. مدل یکی از هشت عمل مجاز (`add` ، `done` ، `undone` ، `del` ، `list` ، `chat` ، `undo` ، `remind`) را به‌صورت یک خط JSON برمی‌گرداند. کد خروجی را اعتبارسنجی می‌کند، روی D1 اجرا می‌کند و پاسخ را می‌فرستد. کل منطق در یک فایل Worker است و هیچ سرور همیشه‌روشنی وجود ندارد.



شکل ۱ — جریان یک پیام. 1) تلگرام با هدر secret به `/webhook` می‌فرستد (بدون هدر درست: ۴۰۳). 2) Worker لیست شماره‌دار کارها و ۶ پیام اخیر را در پرامپت می‌گذارد. 3) مدل یک خط JSON برمی‌گرداند. 4) اعتبارسنجی و اجرا روی D1. 5) پاسخ با `sendMessage` . خط‌چین‌ها: secretها و Cron Trigger. فلش بالا-راست: رونویسی پیام صوتی با Whisper.

- **webhook نه polling:** Worker نمی‌تواند حلقهٔ همیشه‌روشن داشته باشد؛ webhook رویدادمحور است و درخواست بیکار ندارد. خطر باز بودن URL با secret header و پاسخ ۴۰۳ مهار شده است.
- **FaaS:** ترافیک پراکنده (یک کاربر) و scale-to-zero، بدون مدیریت سرور. زمان انتظار برای مدل و D1 جزو CPU time حساب نمی‌شود، پس سقف ۱۰ms پلن رایگان کافی است.
- **D1 نه KV:** به لیست مرتب، فیلتر («باز یا امروزِ انجام‌شده»، «حذف‌نشده»)، تراکنش (`batch`) و سازگاری قوی نیاز داریم. KV نهایی‌سازگار است و کوئری ندارد.
- **state کجاست:** کارها، حافظهٔ گفتگو (۶ پیام اخیر در پرامپت) و شناسهٔ updateهای پردازش‌شده در D1؛ توکن‌ها در Secret store؛ خود Worker بدون state.
- **«امروز» به وقت تهران:** روز تقویمیِ `Asia/Tehran` با `Intl` محاسبه و به‌صورت متن `YYYY-MM-DD` در ستون‌های `created_day` و `done_day` ذخیره می‌شود. کارهای باز همیشه نمایش داده می‌شوند و کارهای انجام‌شده فقط در همان روز تهرانی. زمان‌های دقیق UTC هستند.
- **امنیت:** مقایسهٔ زمان‌ثابتِ secret header (۴۰۳)، لیست سفید با رد بی‌صدا، پاسخ ۲۰۰ فوری با `ctx.waitUntil` و جدول `processed_updates` تا پیام تکراری دوباره اجرا نشود.
- **لایهٔ هوشمند:** پرامپت با لیست شماره‌دار و مثال؛ `parseAction` نوع، طول و شمارهٔ هدف را اعتبارسنجی می‌کند؛ یک تلاش مجدد با شرح خطا؛ سپس پاسخ امنِ «متوجه نشدم» بدون هیچ نوشتنی در DB. حذف نرم (`deleted_at`) و شمارهٔ خارج از محدوده پیام دوستانه می‌گیرد.
- **فیچرهای اضافه:** (الف) **Undo**: هر تغییر در همان batch یک ردیف `action_log` با وضعیتِ قبلِ ردیف‌ها می‌نویسد و «برگردون» آن را بازمی‌گرداند (۲۴ ساعت اخیر). (ب) **یادآور با تاریخ**: مدل تاریخ حساب نمی‌کند، از جدول تقویم ۳۰ روزهٔ پرامپت انتخاب می‌کند و کد اعتبارسنجی می‌کند؛ Cron ساعتی کارهای رسیده را می‌فرستد و `reminded_at` یک‌بارِ ارسال را تضمین می‌کند. (ج) **پیام صوتی**: `getFile` ← دانلود ← Whisper (فارسی) ← همان مسیر متنی؛ متن شنیده‌شده نمایش داده می‌شود تا خطا با Undo جبران شود.
- **یادآور صبحگاهی:** Cron `30 * * * *` (دقیقهٔ ۳۰ هر ساعت UTC، یعنی سر ساعتِ تهران) و بررسی شرطیِ ساعت تهران با `REMINDER_HOUR=8` . Observability روشن و لاگ‌ها JSON ساخت‌یافته‌اند.

۲. تحلیل هزینه در دو مقیاس

فرض‌ها (برآورد، نه اندازه‌گیری): هر کاربر ۲۰ پیام در روز؛ هر پیام یک فراخوانی مدل با ≈ 1,900 توکن ورودی (پرامپت + جدول تقویم + لیست؛ برآورد) و ≈ 50 توکن خروجی با نرخ مدل `gemma-4-26b-a4b-it` (9,091 و 27,273 نیورون در هر میلیون توکن) ⇒ ≈ 18.7 نیورون برای هر پیام؛ هر ویس 10 ثانیه‌ای (Whisper، 46.63 نیورون در دقیقه) ≈ 7.8 نیورون اضافه؛ ≈ 7 نوشتن و ≥ 150 خواندن ردیف D1 برای هر پیام. نرخ‌ها از مستندات Cloudflare (31 اکتبر 2026).

منبع (سقف رایگان)	۱ کاربر	۱۰٬۰۰۰ کاربر
پیام در روز	20	200,000
درخواست Worker (100,000 در روز)	≈ 45 در روز (پیام‌ها + Cron)	≈ 200,000 در روز، **بیش از سقف رایگان** ⇒ پلن Paid با $5 در ماه (10 میلیون درخواست در ماه شامل)
Workers AI (10,000 نیورون در روز)	≈ 375 در روز	≈ 3.7 میلیون در روز ≈ 112 میلیون در ماه ⇒ ≈ $1,230 (با نرخ $0.011 به‌ازای هر هزار نیورون)؛ اگر ۲۰٪ پیام‌ها ویس 10 ثانیه‌ای باشد ≈ $100 بیشتر
D1 نوشتن (100,000 در روز)	≈ 140 در روز	≈ 1.4 میلیون در روز ≈ 42 میلیون در ماه (کمتر از 50 میلیون شامل در Paid)
D1 خواندن (5 میلیون در روز) و فضا (5 GB)	چند هزار ردیف؛ فضا ناچیز	≈ 900 میلیون در ماه (بسیار کمتر از 25 میلیارد شامل)؛ فضا ناچیز
جمع ماهانه	**$0**	**≈ $1,340** (≈ $0.13 برای هر کاربر؛ تقریباً همه‌اش Workers AI)

- در ۱ کاربر، سقف عملی را Workers AI تعیین می‌کند: حدود 10,000 ÷ 18.7 ≈ **530 پیام متنی در روز** (ویس‌ها از همین سهمیه کم می‌شوند).
- جدول تقویم که برای یادآورهای تاریخی به پرامپت اضافه شد، نیورون هر پیام را تقریباً دو برابر کرد؛ این بهای فیچر یادآور است. در ۱۰٬۰۰۰ کاربر هزینه را مدل زبانی تعیین می‌کند، نه Worker یا D1. اهرم‌های کاهش: ارسال فقط 14 روز تقویم، مدل کوچک‌تر، میان‌بُر بدون مدل برای پیام‌های ساده («لیست»)، کوتاه‌کردن پرامپت.
- **این معماری برای ۱۰٬۰۰۰ کاربر کافی نیست.** لازم است: ستون `user_id` و ایندکس، ثبت‌نام و احراز هویت، صف برای بار مدل زبانی، رعایت محدودیت نرخ ارسال تلگرام و پلن پولی.

۳. کجا گیر کردم و چطور باز شد

علامت	بررسی و علت	راه‌حل و درس
`npm install` و `npm test` خطای `ENOENT: package.json` می‌دادند.	با `dir` دیدم پوشهٔ `hermes-mini` فقط `.agents` و `skills-lock.json` دارد. یعنی پوشه‌ای هم‌نام از قبل وجود داشت و پروژه داخلش نبود.	zip را با `Expand-Archive` در مسیر تازه (`C:\Users\Shadi\projects`) باز کردم. درس: قبل از دستور، با `dir` مطمئن شوم در پوشهٔ درست هستم.
`wrangler d1 list` خطای `Invalid TOML` داد (خط 30).	خطا شمارهٔ خط و ستون را نشان می‌داد. موقع تایپ شناسهٔ دیتابیس کیبورد روی فارسی بود: یک حرف «ط» وسط `database_id` افتاده بود و یک کوتیشن اضافه.	شناسه را از خروجی `d1 list` کپی و paste کردم و فایل درست خوانده شد. درس: شناسه‌ها را تایپ نکنم.
`wrangler deploy` خطای `10034` داد: «ایمیل باید تأیید شود».	با `wrangler whoami` دیدم حساب کلادفلر با ایمیلی غیر از ایمیل گیت‌هابم ساخته شده و ایمیل تأیید به همان آدرس رفته بود؛ همین باعث سردرگمی بود.	لینک تأیید را زدم («قبلاً تأیید شده») و deploy دوباره موفق شد. درس: برای فهمیدن حساب فعال، `whoami` را ببینم.
`curl.exe` برای `setWebhook` بعد از ~21 ثانیه timeout می‌داد، با اینکه فیلترشکن روشن بود. `npm run logs` هم مدام قطع‌وصل می‌شد.	فیلتر روی اتصال خودم است، نه روی Worker. احتمالاً ترافیک `curl` از مسیر فیلترشکن عبور نمی‌کرد.	`curl` را با `--proxy` به پورت محلی فیلترشکن وصل کردم و webhook ثبت شد. برای لاگ، داشبورد Observability جایگزین است.
برای جملهٔ مبهم «پروژه رو تحویل بده» ربات می‌گفت «الان	لاگ زنده: `llm_error: empty Workers AI response`. در مستندات دیدم مدل `gemma-4`	`enable_thinking:false` و سقف توکن بیشتر، و خروجی خالی را مثل خروجی نامعتبر دیدم (یک تلاش

نمی‌تونم پیامت رو پردازش کنم»، ولی «سلام» جواب می‌داد.	reasoning است؛ احتمالاً توکن‌ها را صرف فکرکردن می‌کرد و `content` خالی برمی‌گشت.	مجدد، سپس پاسخ امن). پس از deploy همان جمله به‌عنوان یک کار ثبت شد. درس: لاگ را بخوانم و مسیر خطا هرگز بی‌دلیل «تسلیم» نشود.

۴. استفاده از هوش مصنوعی و محدودیت‌ها

روش کار: کد migration ،Worker، تست‌ها و مستندات با Claude ساخته شد. هارنس: ابزارهای فایل و ترمینال؛ MCP کلادفلر برای خواندن مستندات و سقف‌های پلن رایگان؛ Skill پروژه (`hermes-deploy`) برای چک‌لیست deploy؛ و `CLAUDE.md` برای قواعد ثابت پروژه (مثلاً «هیچ secret ای در ریپو»). نصب، ثبت secretها، deploy، ثبت webhook و تست روی ربات واقعی را خودم انجام دادم و خروجی هر مرحله را برای عیب‌یابی به هوش مصنوعی دادم.

محدودیت‌ها: شماره‌گذاری موقعیتی ممکن است بعد از تغییر لیست از دستگاه دیگر جابه‌جا شود. فهم فارسی به مدل بستگی دارد و برای ورودی مبهم سؤال توضیحی می‌پرسد. فهم واقعی «فردا ساعت 9» و رونویسی فارسی Whisper باید روی ربات زنده دیده شود (تست‌های محلی با مدل تقلبی بودند). یادآورها در تست محلی با زمان شبیه‌سازی‌شده تأیید شدند و اجرای زندهٔ Cron باید دیده شود. دقت یادآور ساعتی است. اتصال به «بله» انجام نشد.